Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Securing the Future with Quantum Computing and Cryptography

TGS Ref No: TGS-2024043419

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 11.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
10.0	1 February 2026	Previous release — Learner Guide aligned to the v10 slide deck.	Dr Alfred Ang
11.0	11 August 2026	Realigned to the published TGS-2024043419 outline: Topic 7 is now Quantum Computing Applications and Use Cases. Rewrote all 18 activities with full step-by-step instructions against the reusable Qiskit lab suite, and added the gate reference, algorithm selection guide and PQC quick reference for open-book assessment use.	Dr Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Introduction

This Learner Guide accompanies the WSQ course Securing the Future with Quantum Computing and Cryptography (TGS-2024043419), conducted by Tertiary Infotech Academy Pte Ltd. It provides detailed step-by-step instructions for all 18 hands-on activities, organised by the eight course topics. Every activity maps to one or more of the seven learning outcomes and is completed using Qiskit and the free IBM Quantum Platform.

Use this guide alongside the course slides and the lab source code in the labs/ folder of the course repository. The slides carry the concepts and diagrams; THIS guide carries the commands. The final assessment is open book, and this guide is one of your permitted materials — keep it with you.

Course Learning Outcomes

- LO1: Design mathematical models required for quantum computing applications.
- LO2: Decide on appropriate physical methodologies specific to quantum computing problems.
- LO3: Direct two-level quantum models for quantum computing.
- LO4: Evaluate quantum computing applicability in the business context.
- LO5: Spearhead the application of quantum gates to quantum computing domains.
- LO6: Establish guidelines for selecting quantum computing algorithms.
- LO7: Synthesize quantum computing methodologies to address complex problems in quantum computing.

Before You Start — Environment Setup

What you need

- A laptop running Windows or macOS with Python 3.11 or later installed.
- A free IBM Quantum account — sign up at https://quantum.ibm.com to obtain your API token.
- A free Google account if you prefer to run the activities in Google Colab rather than locally.
- Visual Studio Code or Anaconda for local development (optional if you use Colab).
- An internet connection — the activities submit jobs to real IBM Quantum hardware.

Install the Python packages

Install Qiskit and the supporting libraries. If you are working in Google Colab, run the same pip command in the first notebook cell prefixed with an exclamation mark.

```
pip install qiskit qiskit-ibm-runtime pennylane matplotlib pylatexenc
python-dotenv numpy
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Store your IBM Quantum credentials

Create a file named .env in the labs/ folder and add your IBM Quantum token and instance. The lab scripts load this file automatically with python-dotenv. Never commit this file to version control.

```
IBM_QUANTUM_TOKEN=your_api_token_here

IBM_QUANTUM_INSTANCE=your_instance_here
```

Verify your setup

Run the following to confirm Qiskit is installed and your credentials authenticate correctly. If the backend list prints, you are ready to begin.

```
python -c "import qiskit; print(qiskit.__version__)"

python -c "from qiskit_ibm_runtime import QiskitRuntimeService; \

print([b.name for b in QiskitRuntimeService().backends()])"
```

Conventions used in every activity

- Commands are run from a terminal in the labs/ folder, or in a Google Colab / Jupyter notebook cell.
- Ket notation |0> and |1> is written in plain ASCII in code, and as |0〉 and |1〉 in the explanatory text.
- Each activity states a 'Test it' check — always confirm it before moving on.
- Jobs submitted to real hardware may queue for several minutes; use a simulator while developing.
- Results from real hardware include noise, so expect a small percentage of unexpected outcomes.

Note: If a job queues for too long during class, switch to the local simulator by replacing the backend with AerSimulator() — the circuit logic is identical and results are noise-free.

Topic 01 — Quantum Physics Fundamentals

Postulates of Quantum Mechanics · Mathematics for Quantum Mechanics

Key concepts

- The five postulates — State space, unitary time evolution, observables, the Born rule and wave-function collapse are the axioms every quantum algorithm obeys.
- Hilbert space — Quantum states are unit vectors in a complex inner-product space; a single qubit lives in a 2-dimensional Hilbert space.
- Unitary operators — U†U = I preserves the norm — this is why every quantum gate is reversible and probability is conserved.

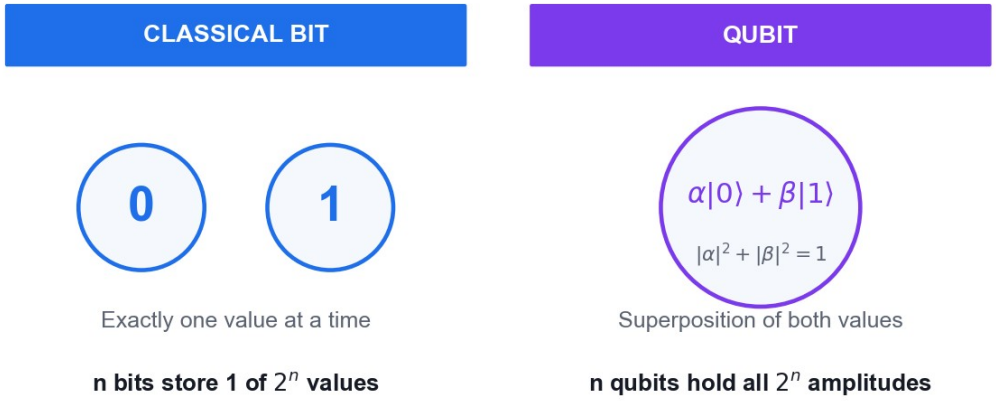
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Hermitian operators — A† = A guarantees real eigenvalues, so Hermitian operators represent physically measurable observables.
- Dirac notation — Bra-ket notation compactly expresses states $|\psi\rangle$, dual vectors $\langle\psi|$, inner products $\langle\varphi|\psi\rangle$ and outer products $|\psi\rangle\langle\varphi|$.
- Tensor products — The state space of a multi-qubit system is the tensor product of the single-qubit spaces — dimension grows as 2^n.

Activity 1 — Linear Algebra for Quantum Mechanics in Python

Learning outcome: LO1 — Design mathematical models required for quantum computing applications.

Goal: Work with the mathematical objects every quantum algorithm is built from: state vectors, inner and outer products, unitary and Hermitian operators, eigenvalues and tensor products — all in NumPy.



A classical bit versus a qubit — the object your linear algebra describes.

What you'll build

A Python notebook that verifies unitarity and Hermiticity, computes eigenvalues/eigenvectors, and builds two-qubit states with the tensor product. (Tools: Python 3.13, NumPy, Google Colab / VS Code.)

Step-by-step

1. Open Google Colab (or VS Code) and create a new Python notebook for this lab.
2. Import NumPy and define the computational basis states |0> = [1,0] and |1> = [0,1] as column vectors.

```
import numpy as np

ket0 = np.array([[1],[0]]); ket1 = np.array([[0],[1]])
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Build a normalised superposition state |+> = (|0> + |1>)/sqrt(2) and confirm its norm is 1.

```
plus = (ket0 + ket1)/np.sqrt(2)

print(np.linalg.norm(plus))   # 1.0
```

4. Define the Pauli-X, Pauli-Y, Pauli-Z and Hadamard matrices as NumPy arrays.

```
X = np.array([[0,1],[1,0]]); Z = np.array([[1,0],[0,-1]])

H = (1/np.sqrt(2))*np.array([[1,1],[1,-1]])
```

5. Verify that H is unitary by checking that H-dagger times H equals the identity.

```
print(np.allclose(H.conj().T @ H, np.eye(2)))   # True
```

6. Verify that X and Z are Hermitian by checking that A-dagger equals A, then confirm their eigenvalues are real.

```
print(np.allclose(X.conj().T, X))

print(np.linalg.eigvals(X))   # [1., -1.]
```

7. Compute the eigenvalues and eigenvectors of the Pauli-Z operator and interpret them as measurement outcomes.

```
vals, vecs = np.linalg.eig(Z)

print(vals); print(vecs)
```

8. Compute the inner product <0|+> and apply the Born rule to obtain the probability of measuring 0.

```
amp = (ket0.conj().T @ plus).item()

print(abs(amp)**2)   # 0.5
```

9. Build the two-qubit state |0> tensor |1> using the Kronecker product and confirm it has 4 components.

```
psi = np.kron(ket0, ket1)

print(psi.T, psi.shape)   # (4, 1)
```

10. Build the two-qubit operator H tensor I and verify that it is also unitary.

```
HI = np.kron(H, np.eye(2))

print(np.allclose(HI.conj().T @ HI, np.eye(4)))
```

Test it

H-dagger H equals the identity, X and Z return real eigenvalues +1 and -1, the Born-rule probability of measuring 0 in |+> is 0.5, and the tensor product produces a 4-component two-qubit state.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The reference implementation for this activity is in labs/linear-algebra/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 02 — Introduction to Quantum Computing Systems

Overview of quantum computers · Commercial systems · Superposition and Entanglement

Key concepts

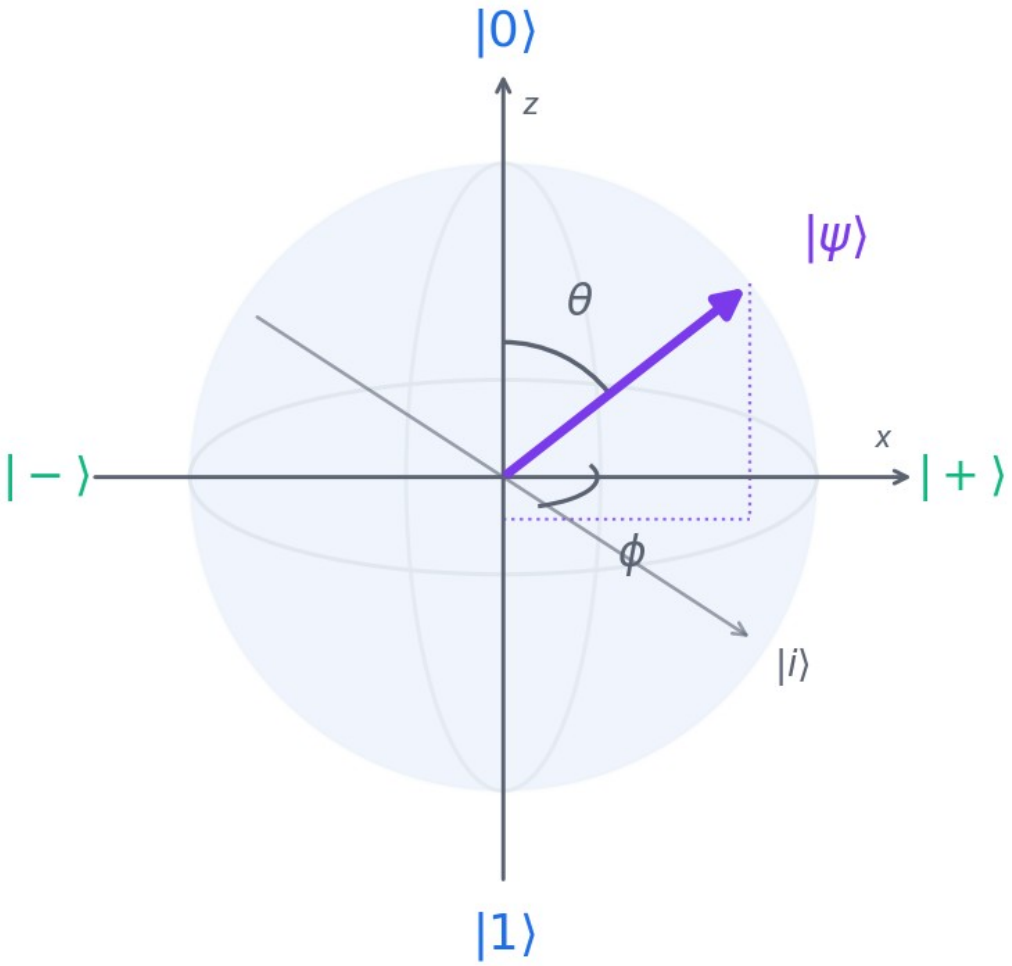
- Qubit vs classical bit — A bit is 0 or 1; a qubit is $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$, allowing superposition of both values.
- Bloch sphere — Every pure single-qubit state maps to a point on the unit sphere — the standard geometric picture of qubit state and rotation.
- Physical qubits — Superconducting (Josephson junction), trapped-ion and topological qubits trade off speed, coherence time and connectivity.
- Commercial platforms — IBM Quantum, Google, IonQ, Rigetti and D-Wave offer cloud access to real quantum processors today.
- Superposition — A quantum register of n qubits holds a superposition of all 2^n basis states simultaneously — the source of quantum parallelism.
- Entanglement — Entangled states cannot be written as a product of single-qubit states; measuring one qubit instantly correlates with the other.

Activity 2 — Visualising a Qubit on the Bloch Sphere

Learning outcome: LO2 — Decide on appropriate physical methodologies specific to quantum computing problems.

Goal: Build single-qubit states in Qiskit and plot them on the Bloch sphere, so you can see how gates rotate a qubit and what superposition and phase actually look like geometrically.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



The Bloch sphere: theta sets the measurement probability, phi sets the phase.

What you'll build

A set of Bloch-sphere plots showing |0>, |1>, |+>, |-> and an arbitrary Ry-rotated state. (Tools: Qiskit 2.x, matplotlib, Google Colab.)

Step-by-step

1. Install Qiskit with visualisation support in your Colab notebook or local environment.

```
pip install 'qiskit[visualization]' matplotlib pylatexenc
```

2. Import Statevector and plot_bloch_multivector from Qiskit.

```
from qiskit.quantum_info import Statevector
```

```
from qiskit.visualization import plot_bloch_multivector
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

3. Create a one-qubit circuit left in the |0> state and plot it — the vector points to the north pole.

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(1)

plot_bloch_multivector(Statevector(qc))
```

4. Apply an X gate to flip the qubit to |1> and re-plot — the vector now points to the south pole.

```
qc = QuantumCircuit(1); qc.x(0)

plot_bloch_multivector(Statevector(qc))
```

5. Apply a Hadamard gate to reach the |+> state and observe the vector lying on the equator along +x.

```
qc = QuantumCircuit(1); qc.h(0)

plot_bloch_multivector(Statevector(qc))
```

6. Apply H then Z to reach |-> and confirm the vector points along -x.

```
qc = QuantumCircuit(1); qc.h(0); qc.z(0)

plot_bloch_multivector(Statevector(qc))
```

7. Apply an Ry rotation of pi/4 to create an arbitrary superposition and read off its amplitudes.

```
import numpy as np

qc = QuantumCircuit(1); qc.ry(np.pi/4, 0)

print(Statevector(qc))
```

8. Apply an S gate after H and observe that a phase gate rotates the vector around the z-axis without changing measurement probabilities in the Z basis.

```
qc = QuantumCircuit(1); qc.h(0); qc.s(0)

plot_bloch_multivector(Statevector(qc))
```

9. Record which gates move the vector between poles and which rotate it around the equator.

Test it

|0> sits at the north pole, X moves it to the south pole, H places it on the equator at +x, and the S gate rotates it around the equator while leaving the Z-basis probabilities at 50/50.

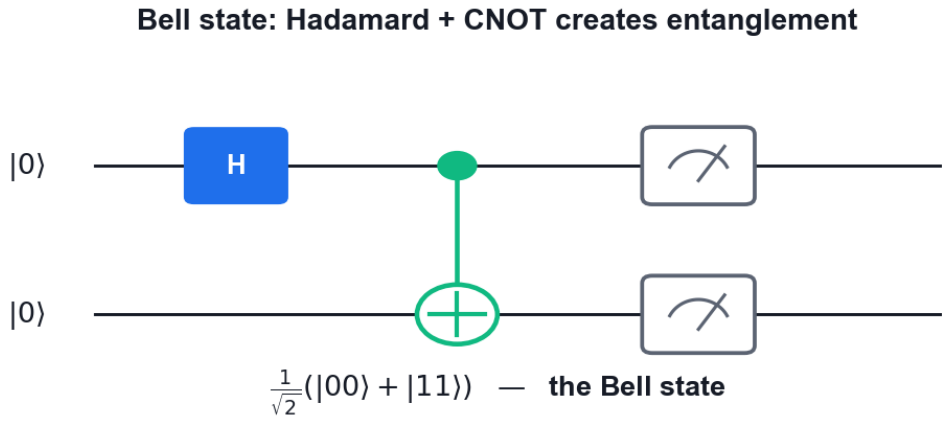
Note: The reference implementation for this activity is in labs/bloch-sphere/. Read the source before you run it — understanding the circuit matters more than the output.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 3 — Superposition and Entanglement on IBM Quantum

Learning outcome: LO2, LO4 — Evaluate quantum computing applicability in the business context.

Goal: Create your IBM Quantum account, build a superposition circuit and a two-qubit Bell state, and run them on a real quantum processor to see genuine quantum randomness and correlation in the results.



The Bell state circuit you will build and run on IBM Quantum hardware.

What you'll build

A Bell state circuit executed on real IBM Quantum hardware, with a measured histogram showing the |00> and |11> correlation. (Tools: IBM Quantum Platform, Qiskit Runtime, Python 3.13.)

Step-by-step

1. Sign up for a free IBM Quantum account and copy your API token from the dashboard.

```
# https://quantum.ibm.com
```

2. Store the token and instance in a .env file in the labs folder so the scripts can authenticate.

```
IBM_QUANTUM_TOKEN=your_api_token_here

IBM_QUANTUM_INSTANCE=your_instance_here
```

3. Install the Qiskit runtime packages in your environment.

```
pip install qiskit qiskit-ibm-runtime python-dotenv
```

4. Build a one-qubit superposition circuit with a Hadamard gate and a measurement.

```
from qiskit import QuantumCircuit
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
qc = QuantumCircuit(1,1); qc.h(0); qc.measure(0,0)
```

5. Build the Bell state circuit: Hadamard on qubit 0, then CNOT from qubit 0 to qubit 1.

```
bell = QuantumCircuit(2,2)

bell.h(0); bell.cx(0,1); bell.measure([0,1],[0,1])
```

6. Connect to IBM Quantum and select the least busy real backend with at least 2 qubits.

```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService()

backend = service.least_busy(operational=True, simulator=False,
min_num_qubits=2)
```

7. Transpile the Bell circuit for the chosen backend so it uses only that device's native gates.

```
from qiskit import transpile

tqc = transpile(bell, backend, optimization_level=1)
```

8. Submit the job with the SamplerV2 primitive using 1024 shots and wait for the result.

```
from qiskit_ibm_runtime import SamplerV2 as Sampler

job = Sampler(mode=backend).run([tqc], shots=1024)

print(job.job_id())
```

9. Plot the measurement histogram and compare it against the ideal 50/50 split.

```
from qiskit.visualization import plot_histogram
```

10. Explain why a small number of |01> and |10> counts appear on real hardware but never in the ideal simulation.

Test it

The superposition circuit gives roughly 50% |0> and 50% |1>; the Bell circuit gives roughly 50% |00> and 50% |11> with only a small percentage of |01> and |10> counts caused by hardware noise.

Note: The reference implementation for this activity is in labs/bell-state/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 03 — Quantum Gates and Circuits

Single Qubit Gates · Multi Qubit Gates · Bell and GHZ States · Phase Kickback

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

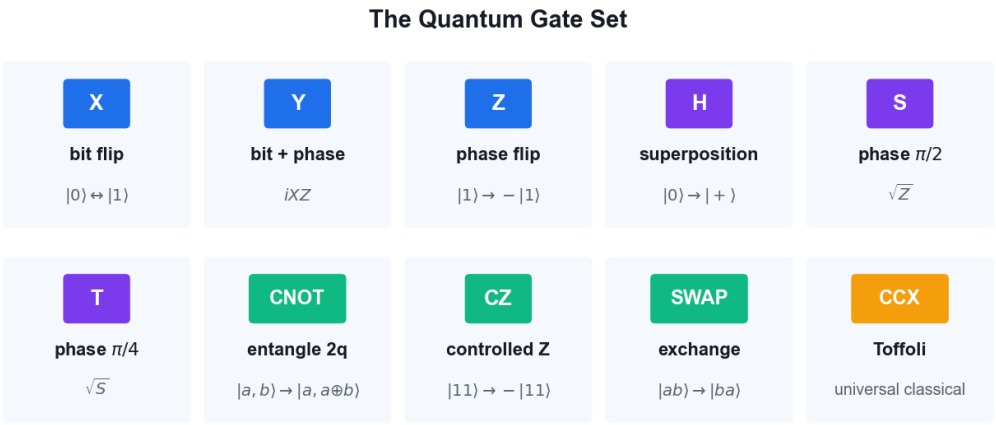
Key concepts

- Pauli gates — X (bit flip), Y and Z (phase flip) are self-inverse, Hermitian and unitary — the building blocks of single-qubit logic.
- Hadamard gate — H maps $|0\rangle \to |+\rangle$ and $|1\rangle \to |-\rangle$, creating the equal superposition that starts nearly every quantum algorithm.
- Rotation and phase gates — Rx, Ry, Rz rotate the Bloch vector; S and T are special cases of the phase gate P(θ).
- Multi-qubit gates — CNOT, CZ, controlled-phase, SWAP and Toffoli (CCNOT) create the entanglement that makes quantum computing powerful.
- Bell and GHZ states — H followed by CNOT produces a maximally entangled Bell pair; extending to n qubits gives the GHZ state.
- Phase kickback — A controlled operation on an eigenstate transfers the eigenvalue phase back to the control qubit — the engine of Deutsch, BV and QPE.

Activity 4 — Building Circuits in IBM Quantum Composer

Learning outcome: LO5 — Spearhead the application of quantum gates to quantum computing domains.

Goal: Use the drag-and-drop IBM Quantum Composer to build single- and multi-qubit circuits, watch the state vector and probability panels update live, and export the circuit as OpenQASM.



The gate palette available in IBM Quantum Composer.

What you'll build

A set of Composer circuits (X, H, CNOT, Bell state) with their statevector output, exported as OpenQASM 3 code. (Tools: IBM Quantum Composer, OpenQASM 3.)

Step-by-step

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

1. Open IBM Quantum Composer in your browser and sign in with your IBM Quantum account.

```
# https://quantum.ibm.com/composer
```

2. Create a new circuit with 2 qubits and open the Statevector and Probabilities visualisation panels.
3. Drag an X gate onto qubit 0 and observe the state change from |00> to |01> in the probability panel.
4. Remove the X gate, drag a Hadamard gate onto qubit 0 and observe the equal superposition appear.
5. Add a CNOT gate with qubit 0 as control and qubit 1 as target to create the Bell state.
6. Confirm the probability panel shows only |00> and |11> at 50% each — the signature of entanglement.
7. Switch to the OpenQASM view and copy the generated code for your circuit.

```
OPENQASM 3;

include "stdgates.inc";

qubit[2] q;

h q[0];

cx q[0], q[1];
```

8. Load the provided bell-state QASM file from the labs folder into Composer and confirm it reproduces the same circuit.
9. Experiment with Z, S and T gates after the Hadamard and note which ones change the probability panel and which only change the phase.

Test it

The Composer probability panel shows a 50/50 split between |00> and |11> for the Bell circuit, and the exported OpenQASM contains an h instruction followed by a cx instruction.

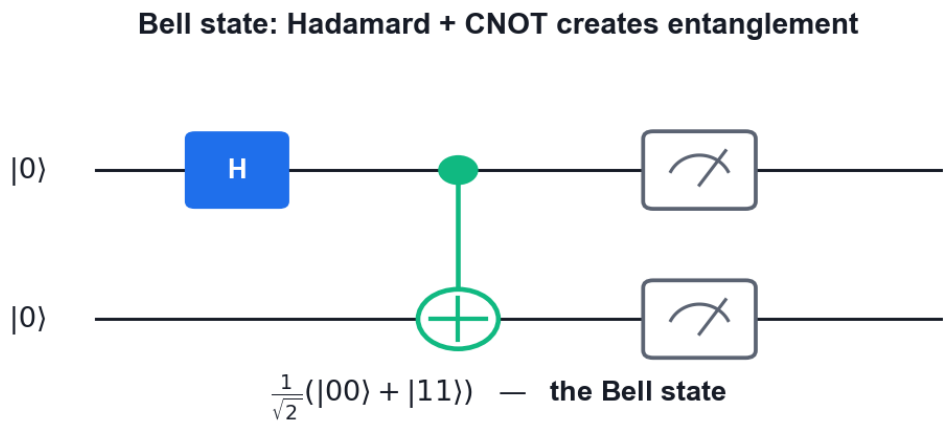
Note: The reference implementation for this activity is in labs/quantum-composer/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 5 — Quantum Gates and Bell/GHZ States with PennyLane

Learning outcome: LO3, LO5 — Direct two-level quantum models and apply quantum gates.

Goal: Use PennyLane's quantum-function and QNode model to apply the full single- and multi-qubit gate set, then construct Bell and GHZ entangled states and verify their measurement correlations.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Hadamard followed by CNOT — the Bell state you build in PennyLane.

What you'll build

A PennyLane script that draws each circuit, returns the state vector, and confirms the Bell and GHZ correlations. (Tools: PennyLane, Python 3.13, matplotlib.)

Step-by-step

1. Install PennyLane in your environment.

```
pip install pennylane
```

2. Create a 2-qubit default.qubit device to run your circuits on.

```
import pennylane as qml

dev = qml.device('default.qubit', wires=2)
```

3. Write a quantum function that applies a Pauli-X to wire 0 and returns the state, then wrap it in a QNode.

```
@qml.qnode(dev)

def circuit():

    qml.PauliX(0)

    return qml.state()
```

4. Print the circuit diagram using PennyLane's draw function to confirm the gate placement.

```
print(qml.draw(circuit)())
```

5. Build the Bell state by applying a Hadamard to wire 0 followed by a CNOT from wire 0 to wire 1.

```
@qml.qnode(dev)

def bell():
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
    qml.Hadamard(0); qml.CNOT([0,1])

    return qml.state()
```

6. Print the resulting state vector and confirm the amplitudes are 0.707 on |00> and |11> only.

```
print(bell())   # [0.707, 0, 0, 0.707]
```

7. Change the device to 3 wires and build the GHZ state with a Hadamard and two CNOT gates.

```
dev3 = qml.device('default.qubit', wires=3)

@qml.qnode(dev3)

def ghz():

    qml.Hadamard(0); qml.CNOT([0,1]); qml.CNOT([1,2])

    return qml.state()
```

8. Verify the GHZ state has amplitude only on |000> and |111>.

```
print(ghz())
```

9. Replace the state return with qml.probs to obtain measurement probabilities instead of amplitudes.

```
return qml.probs(wires=[0,1])
```

10. Demonstrate phase kickback: apply a controlled-Z with the target in the |-> eigenstate and observe the phase appear on the control qubit.

Test it

The Bell state returns amplitudes of about 0.707 on |00> and |11> and zero elsewhere; the GHZ state returns amplitudes only on |000> and |111>; the phase-kickback circuit shows the eigenvalue phase transferred to the control qubit.

Note: The reference implementation for this activity is in labs/pennylane-gates/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 04 — Quantum Protocols

No-Cloning Theorem · Teleportation · Superdense Coding · BB84 QKD · CHSH Game

Key concepts

- No-cloning theorem — No unitary can copy an arbitrary unknown quantum state — proved by linearity, and the foundation of quantum key security.

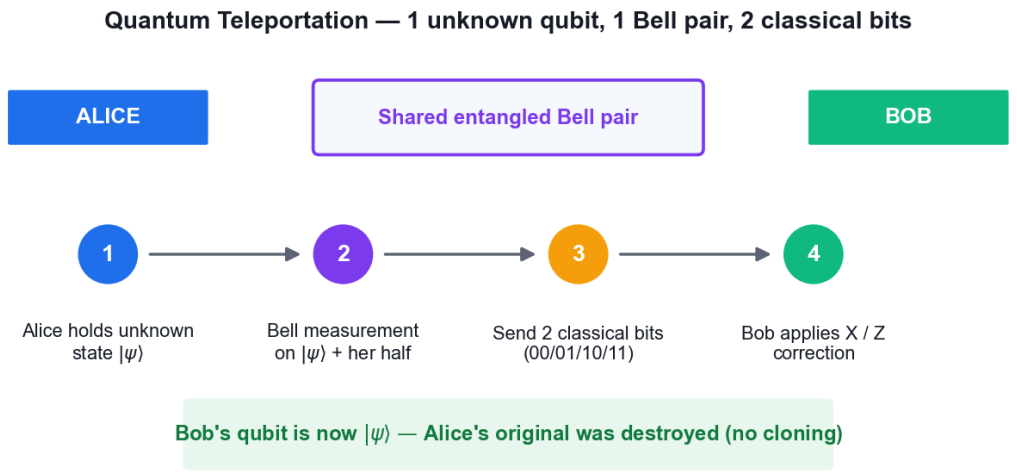
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Quantum teleportation — Transfers an unknown qubit state using one shared Bell pair plus two classical bits — no faster-than-light signalling.
- Superdense coding — The dual of teleportation: send two classical bits by transmitting a single qubit of a pre-shared entangled pair.
- BB84 QKD — Alice and Bob build a shared secret key using random bases; eavesdropping disturbs the states and raises the error rate.
- Eavesdropping detection — An intercept-resend attack introduces roughly a 25% error rate in the sifted key, revealing Eve's presence.
- CHSH game — A quantum strategy using entanglement wins ~85% of the time versus the classical maximum of 75%, disproving local hidden variables.

Activity 6 — Quantum Teleportation

Learning outcome: LO3, LO7 — Synthesize quantum methodologies to address complex problems.

Goal: Implement the three-qubit teleportation protocol: entangle a shared pair, perform a Bell measurement on the unknown state, and apply the conditional corrections that recreate the original state on Bob's qubit.



The four stages of the quantum teleportation protocol.

What you'll build

A teleportation circuit run on IBM Quantum hardware that reconstructs an arbitrary input state at the receiving qubit. (Tools: Qiskit 2.x, IBM Quantum Runtime, OpenQASM.)

Step-by-step

1. Open the quantum-teleportation lab folder and review main.py before running it.

```
cd labs/quantum-teleportation && cat main.py
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

2. Create a 3-qubit circuit: q0 carries the state to teleport, q1 and q2 form the entangled pair.

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(3, 3)
```

3. Prepare an arbitrary state on q0 using a rotation so you have something non-trivial to teleport.

```
import numpy as np

qc.ry(np.pi/4, 0)
```

4. Create the Bell pair between q1 and q2 with a Hadamard and a CNOT.

```
qc.h(1); qc.cx(1, 2)
```

5. Perform Alice's Bell measurement: CNOT from q0 to q1, then a Hadamard on q0.

```
qc.cx(0, 1); qc.h(0)
```

6. Measure q0 and q1 into the classical register — these are the two classical bits sent to Bob.

```
qc.measure([0, 1], [0, 1])
```

7. Apply Bob's conditional corrections: an X gate if bit 1 is set and a Z gate if bit 0 is set.

```
with qc.if_test((1, 1)): qc.x(2)

with qc.if_test((0, 1)): qc.z(2)
```

8. Measure q2 and run the circuit on the least busy IBM Quantum backend.

```
python labs/quantum-teleportation/main.py
```

9. Compare the measured distribution on q2 against the original state prepared on q0.
10. Explain why teleportation does not violate the no-cloning theorem — the original state is destroyed by the Bell measurement.

Test it

The measurement statistics on Bob's qubit q2 match the amplitudes of the state originally prepared on q0, within hardware noise, and the original state no longer exists on q0.

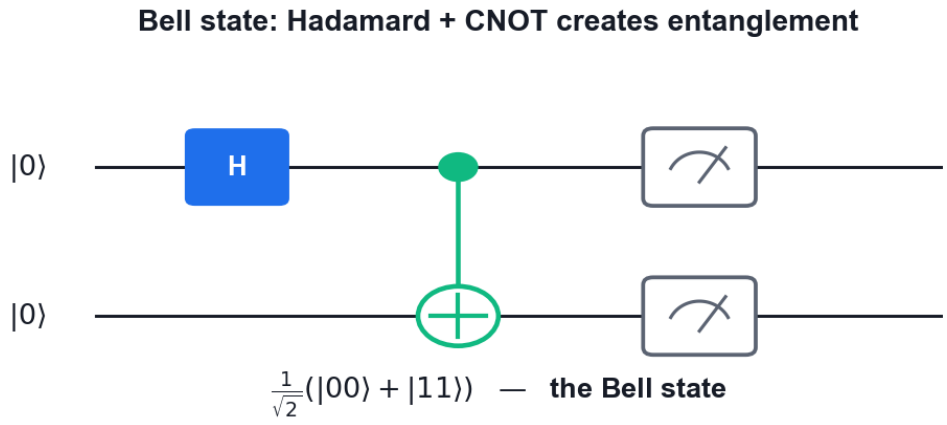
Note: The reference implementation for this activity is in labs/quantum-teleportation/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 7 — Superdense Coding Protocol

Learning outcome: LO3, LO7 — Direct two-level quantum models for quantum computing.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Goal: Implement the dual of teleportation: transmit two classical bits by sending only a single qubit of a pre-shared entangled pair, encoding the message with the I, X, Z and XZ operations.



Superdense coding starts from the same shared Bell pair.

What you'll build

A superdense coding circuit that correctly transmits all four two-bit messages 00, 01, 10 and 11. (Tools: Qiskit 2.x, IBM Quantum Runtime.)

Step-by-step

1. Open the superdense-coding lab folder and review the encoding table in main.py.

```
cd labs/superdense-coding && cat main.py
```

2. Create a 2-qubit circuit and build the shared Bell pair with a Hadamard and CNOT.

```
qc = QuantumCircuit(2, 2)
```

```
qc.h(0); qc.cx(0, 1)
```

3. Encode the message 00 by applying the identity — that is, doing nothing to Alice's qubit.
4. Encode the message 01 by applying an X gate to Alice's qubit.

```
qc.x(0)
```

5. Encode the message 10 by applying a Z gate to Alice's qubit.

```
qc.z(0)
```

6. Encode the message 11 by applying both Z and X to Alice's qubit.

```
qc.z(0); qc.x(0)
```

7. Decode by reversing the entangling operation: CNOT then Hadamard, and measure both qubits.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
qc.cx(0, 1); qc.h(0)

qc.measure([0,1],[0,1])
```

8. Run all four encodings on IBM Quantum hardware and record the measured bit strings.

```
python labs/superdense-coding/main.py
```

9. Confirm each encoding decodes to its intended two-bit message with high probability.

Test it

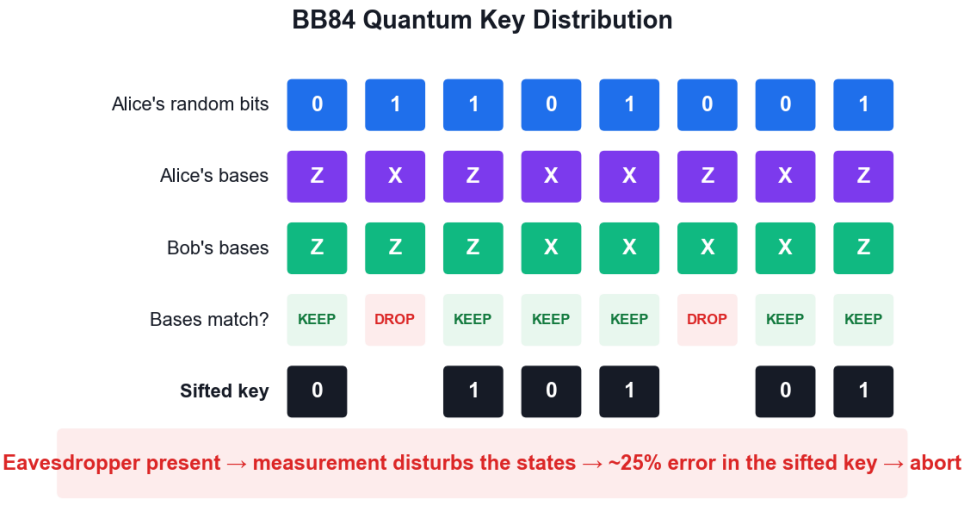
Each of the four encodings decodes to its intended message — identity gives 00, X gives 01, Z gives 10 and ZX gives 11 — demonstrating two classical bits carried by one transmitted qubit.

Note: The reference implementation for this activity is in labs/superdense-coding/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 8 — BB84 Quantum Key Distribution with Eavesdropping Detection

Learning outcome: LO7 — Synthesize quantum methodologies to address complex problems.

Goal: Run the BB84 protocol end to end: random bit and basis selection, quantum transmission, basis sifting and key extraction — then repeat it with an eavesdropper present and measure the error rate that exposes the attack.



BB84: basis sifting, and the error rate that exposes an eavesdropper.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A working BB84 implementation producing a shared secret key, plus a comparison run showing the ~25% error rate introduced by an intercept-resend eavesdropper. (Tools: Qiskit 2.x, IBM Quantum Runtime, Python.)

Step-by-step

1. Open the bb84 lab folder — it contains a run without eavesdropping and a run with eavesdropping.

```
cd labs/bb84 && ls
```

2. Review how Alice generates a random bit string and a random basis string for each qubit.

```
cat bb84-without-eavesdropping/main.py
```

3. Understand the encoding rule: in the Z basis send |0> or |1>; in the X basis send |+> or |->.
4. Note how Bob independently chooses a random measurement basis for each incoming qubit.
5. Run the protocol without an eavesdropper and observe the sifted key after basis reconciliation.

```
python bb84-without-eavesdropping/main.py
```

6. Confirm that where Alice and Bob used the same basis, their bits agree almost perfectly.
7. Now run the version with Eve performing an intercept-resend attack on the channel.

```
python bb84-with-eavesdropping/main.py
```

8. Compare the error rate in the sifted key between the two runs.
9. Explain why Eve's measurement disturbs the state and produces roughly a 25% error rate in the sifted key.
10. Discuss how Alice and Bob decide to discard the key when the measured error rate exceeds their security threshold.

Test it

Without an eavesdropper the sifted key error rate is near zero (bounded only by hardware noise); with the intercept-resend eavesdropper the error rate rises to roughly 25%, which is the signal to abort and discard the key.

Note: The reference implementation for this activity is in labs/bb84/. Read the source before you run it — understanding the circuit matters more than the output.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Topic 05 — Quantum Algorithms

Deutsch · Deutsch-Jozsa · Bernstein-Vazirani · Grover · Quantum Fourier Transform

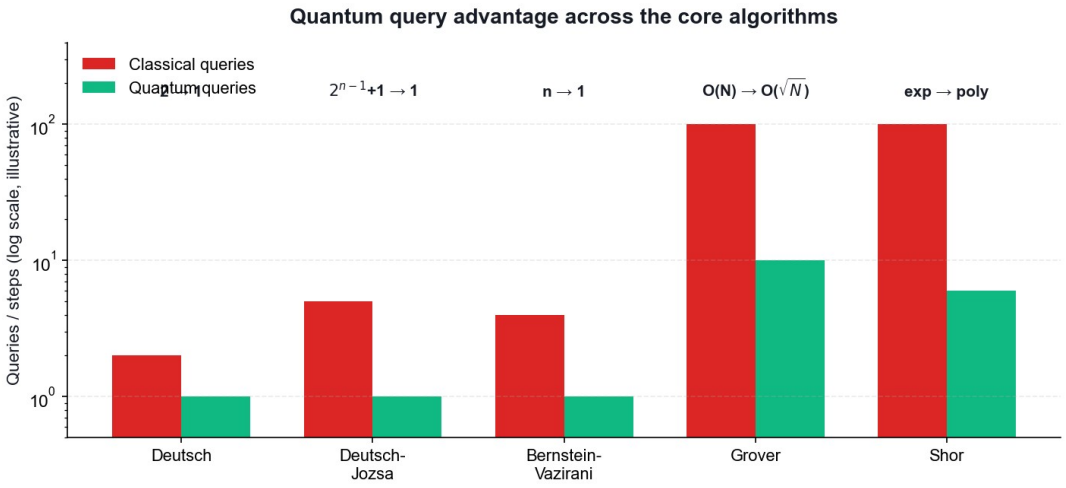
Key concepts

- The query (oracle) model — Algorithms are compared by how many times they query a black-box function — the standard measure of quantum advantage.
- Deutsch algorithm — Decides whether f:{0,1}→{0,1} is constant or balanced with ONE query; classically two are required.
- Deutsch-Jozsa — Extends the idea to n bits — one query versus $2^{n-1}+1$ classical queries in the worst case: exponential separation.
- Bernstein-Vazirani — Recovers an n-bit hidden string s from f(x) = s·x (mod 2) in a single query instead of n.
- Grover search — Amplitude amplification finds a marked item in an unstructured database in O(√N) instead of O(N) — a quadratic speedup.
- Quantum Fourier Transform — The quantum analogue of the DFT runs in O(n²) gates and underpins phase estimation and Shor's algorithm.

Activity 9 — Deutsch and Deutsch-Jozsa Algorithms

Learning outcome: LO6 — Establish guidelines for selecting quantum computing algorithms.

Goal: Implement the first algorithms to show a provable quantum advantage: decide whether a black-box function is constant or balanced with a single query, then generalise from one input bit to n.



Query advantage of Deutsch and Deutsch-Jozsa over classical methods.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Deutsch and Deutsch-Jozsa circuits with all four oracle types, run on IBM Quantum hardware, correctly classifying each function in one query. (Tools: Qiskit 2.x, IBM Quantum Runtime, OpenQASM.)

Step-by-step

1. Open the deutsch-algorithm lab and review the four oracle constructions in main.py.

```
cd labs/deutsch-algorithm && cat main.py
```

2. Understand the oracle convention |x>|y> maps to |x>|y XOR f(x)>, which keeps the operation reversible.
3. Build the Deutsch circuit: initialise q1 to |1>, apply Hadamard to both qubits.

```
qc = QuantumCircuit(2, 1)

qc.x(1); qc.h(0); qc.h(1)
```

4. Apply the oracle for the balanced_identity case, which is a single CNOT.

```
qc.cx(0, 1)
```

5. Apply a final Hadamard to q0 and measure it — 0 means constant, 1 means balanced.

```
qc.h(0); qc.measure(0, 0)
```

6. Run the circuit against all four oracles and confirm each is classified correctly in one query.

```
python labs/deutsch-algorithm/main.py
```

7. Explain how phase kickback makes the oracle mark the input register rather than the output register.
8. Move to the deutsch-jozsa lab and review how the oracle generalises to n input qubits.

```
cd ../deutsch-jozsa-algorithm && cat main.py
```

9. Run the Deutsch-Jozsa algorithm and confirm that an all-zeros measurement means constant and any other result means balanced.

```
python main.py
```

10. Compare the single quantum query against the 2^(n-1)+1 classical queries needed in the worst case.

Test it

The Deutsch circuit measures 0 for both constant oracles and 1 for both balanced oracles; Deutsch-Jozsa returns the all-zeros string for a constant function and a non-zero string for a balanced function — each using exactly one oracle query.

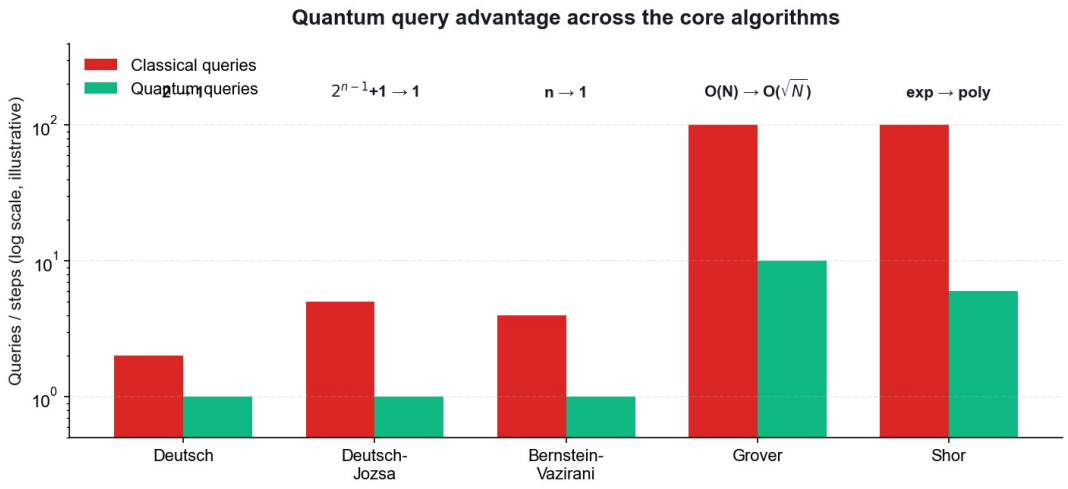
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The reference implementation for this activity is in labs/deutsch-algorithm/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 10 — Bernstein-Vazirani Algorithm

Learning outcome: LO6 — Establish guidelines for selecting quantum computing algorithms.

Goal: Recover an n-bit hidden string in a single oracle query. Classically you would need n queries — one per bit — making this a clean demonstration of linear-to-constant query speedup.



Bernstein-Vazirani recovers an n-bit string in one query instead of n.

What you'll build

A Bernstein-Vazirani circuit that recovers a chosen hidden string such as 1011 in one query on real hardware. (Tools: Qiskit 2.x, IBM Quantum Runtime.)

Step-by-step

1. Open the bernstein-vazirani-algorithm lab and choose a hidden string, for example s = 1011.

```
cd labs/bernstein-vazirani-algorithm && cat main.py
```

2. Create a circuit with n input qubits plus one ancilla, and set the ancilla to |1>.

```
qc = QuantumCircuit(5, 4)

qc.x(4)
```

3. Apply Hadamard gates to every qubit including the ancilla.

```
qc.h(range(5))
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Build the oracle by placing a CNOT from input qubit i to the ancilla wherever bit i of s is 1.

```
for i, bit in enumerate(reversed('1011')):

    if bit == '1': qc.cx(i, 4)
```

5. Apply Hadamard gates to the input qubits again to complete the interference step.

```
qc.h(range(4))
```

6. Measure the input register and run the circuit on IBM Quantum hardware.

```
python labs/bernstein-vazirani-algorithm/main.py
```

7. Confirm the most frequent measurement outcome is exactly the hidden string s.
8. Repeat with a different hidden string and verify the algorithm recovers it in one query again.
9. Explain why n classical queries are required to learn n bits, while the quantum circuit needs only one.

Test it

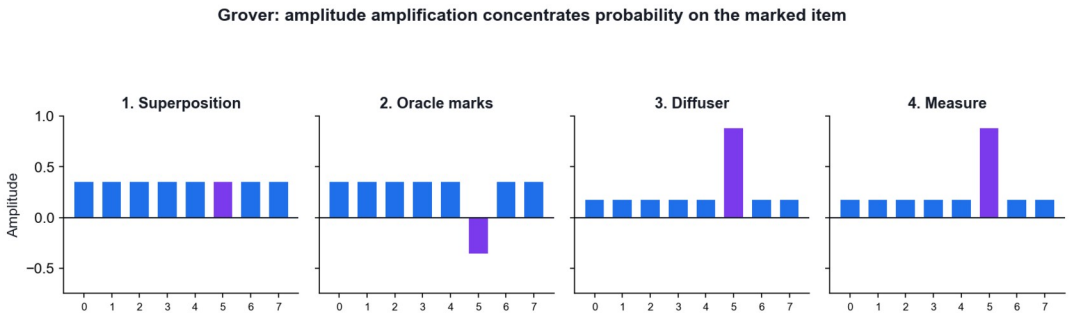
The most frequent measured bit string equals the hidden string s that was encoded in the oracle, recovered with a single oracle query regardless of the length of s.

Note: The reference implementation for this activity is in labs/bernstein-vazirani-algorithm/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 11 — Grover's Search Algorithm

Learning outcome: LO6, LO7 — Select algorithms and synthesize quantum methodologies.

Goal: Implement amplitude amplification to find a marked item in an unstructured search space, using the phase oracle and the diffuser, and see the quadratic O(sqrt(N)) speedup for 2-qubit and 3-qubit searches.



Grover: the oracle marks, the diffuser amplifies.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

What you'll build

Grover circuits for 2 and 3 qubits that amplify the marked state to a high measurement probability on real hardware. (Tools: Qiskit 2.x, IBM Quantum Runtime, OpenQASM.)

Step-by-step

1. Open the grover-algorithm lab, which contains both the 2-qubit and 3-qubit versions.

```
cd labs/grover-algorithm && cat main.py
```

2. Initialise the register in an equal superposition of all N = 2^n basis states with Hadamards.

```
qc = QuantumCircuit(2, 2)

qc.h([0, 1])
```

3. Build the phase oracle that flips the sign of the marked state — for |11> this is a controlled-Z.

```
qc.cz(0, 1)
```

4. Build the diffuser: Hadamards, X gates, a controlled-Z, then X gates and Hadamards again.

```
qc.h([0,1]); qc.x([0,1]); qc.cz(0,1); qc.x([0,1]); qc.h([0,1])
```

5. Measure the register and run the 2-qubit search — one Grover iteration is optimal here.

```
python labs/grover-algorithm/main.py
```

6. Confirm the marked state is measured with close to 100% probability after a single iteration.
7. Move to the 3-qubit case and compute the optimal number of iterations as about pi/4 times sqrt(N).
8. Run the 3-qubit search with 2 iterations and confirm the marked state dominates the histogram.
9. Demonstrate over-rotation by applying too many iterations and observing the success probability fall.
10. Compare the O(sqrt(N)) quantum queries against the O(N) classical brute-force search.

Test it

For 2 qubits the marked state is measured with near-certainty after one Grover iteration; for 3 qubits it dominates the histogram after two iterations; applying too many iterations visibly reduces the success probability.

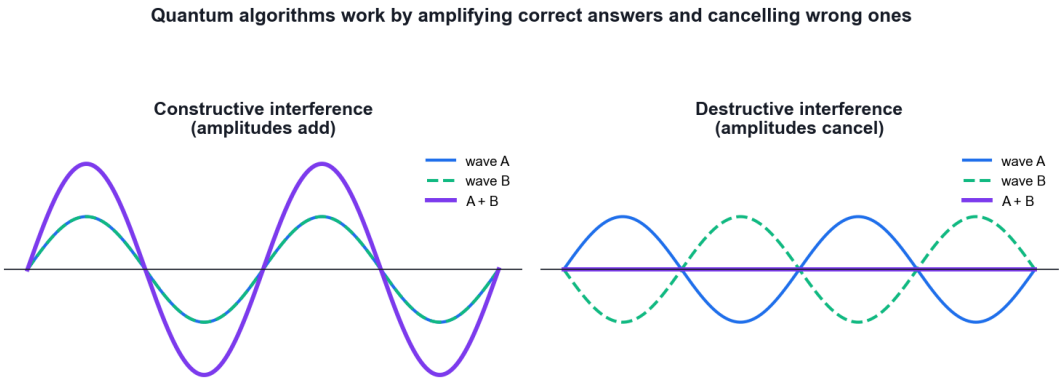
Note: The reference implementation for this activity is in labs/grover-algorithm/. Read the source before you run it — understanding the circuit matters more than the output.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Activity 12 — Quantum Fourier Transform

Learning outcome: LO6, LO7 — Establish guidelines for selecting quantum computing algorithms.

Goal: Build the QFT circuit from Hadamard and controlled-phase gates, verify it against the classical DFT, and understand why it is the engine behind phase estimation and Shor's algorithm.



The QFT works by controlled interference of amplitudes.

What you'll build

QFT circuits for 3, 4 and 5 qubits, validated against a classical DFT reference and run on IBM Quantum hardware. (Tools: Qiskit 2.x, NumPy, IBM Quantum Runtime, OpenQASM.)

Step-by-step

1. Open the quantum-fourier-transform lab, which provides 3, 4 and 5 qubit versions.

```
cd labs/quantum-fourier-transform && cat main.py
```

2. Apply a Hadamard to the most significant qubit to begin the QFT construction.

```
qc = QuantumCircuit(3)

qc.h(2)
```

3. Apply controlled-phase rotations of pi/2, pi/4 and so on from each lower qubit to the current one.

```
import numpy as np

qc.cp(np.pi/2, 1, 2); qc.cp(np.pi/4, 0, 2)
```

4. Repeat the Hadamard and controlled-phase pattern for each remaining qubit in turn.

```
qc.h(1); qc.cp(np.pi/2, 0, 1); qc.h(0)
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Add the final SWAP gates that reverse the qubit order to match the standard QFT convention.

```
qc.swap(0, 2)
```

6. Compare your circuit against Qiskit's built-in QFT to confirm they are equivalent.

```
from qiskit.circuit.library import QFT

print(QFT(3).decompose())
```

7. Verify the output against a classical NumPy FFT of the same input state vector.

```
np.fft.fft(state) / np.sqrt(len(state))
```

8. Run the 3-qubit QFT on IBM Quantum hardware and inspect the transpiled circuit depth.

```
python labs/quantum-fourier-transform/main.py
```

9. Count the gates and confirm the QFT uses O(n^2) gates versus O(N log N) for the classical FFT.

Test it

Your hand-built QFT circuit produces the same state vector as Qiskit's built-in QFT and matches a normalised classical FFT of the same input, using on the order of n^2 gates.

Note: The reference implementation for this activity is in labs/quantum-fourier-transform/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 06 — Quantum Cryptography and Quantum Safety

Classical Cryptography · The Quantum Threat · Shor Algorithm · Post-Quantum Cryptography

Key concepts

- Goals of cryptography — Confidentiality, integrity, authentication and non-repudiation are the four properties every scheme must address.
- Classical schemes — Symmetric (AES), asymmetric (RSA, ECC) and hash functions (SHA-256) rest on problems that are hard for classical computers.
- The quantum threat — Shor's algorithm breaks RSA and ECC in polynomial time; Grover halves the effective key length of symmetric ciphers.
- Shor's algorithm — Reduces factoring to period-finding, then uses the QFT to extract the period — turning a hard problem into an easy one.
- Harvest now, decrypt later — Adversaries can record encrypted traffic today and decrypt it once a cryptographically relevant quantum computer exists.

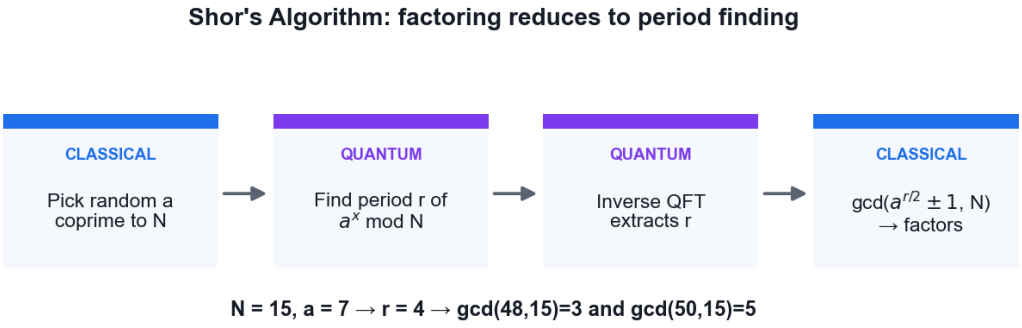
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Post-quantum cryptography — NIST has standardised ML-KEM (Kyber), ML-DSA (Dilithium), SLH-DSA (SPHINCS+) and HQC as quantum-resistant replacements.

Activity 13 — Shor's Algorithm and the Quantum Threat to RSA

Learning outcome: LO7 — Synthesize quantum methodologies to address complex problems.

Goal: See how factoring reduces to period finding, run the quantum period-finding circuit that sits at the heart of Shor's algorithm, and recover the factors of N = 15 from the measured period.



Shor's algorithm: factoring reduced to quantum period finding.

What you'll build

A period-finding circuit whose measured output yields the period r, from which the factors of N = 15 are classically recovered. (Tools: Qiskit 2.x, IBM Quantum Runtime, Python, OpenQASM.)

Step-by-step

1. Open the shor-algorithm lab and review the period-finding construction in main.py.

```
cd labs/shor-algorithm && cat main.py
```

2. Understand the classical reduction: to factor N, pick a random a coprime to N and find the period r of a^x mod N.
3. Verify the reduction by hand for N = 15 and a = 7, listing the powers of 7 mod 15 until the sequence repeats.

```
# 7, 4, 13, 1, 7, ...  -> r = 4
```

4. Build the counting register in superposition with Hadamard gates.

```
qc.h(range(n_count))
```

5. Apply the controlled modular-exponentiation operations that encode a^x mod N.
6. Apply the inverse QFT to the counting register to extract the period information.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
qc.append(QFT(n_count, inverse=True), range(n_count))
```

7. Measure the counting register and run the circuit on IBM Quantum hardware.

```
python labs/shor-algorithm/main.py
```

8. Use the continued-fractions method on the measured phase to recover the period r.
9. Compute the factors as gcd(a^(r/2) +/- 1, N) and confirm you obtain 3 and 5.

```
from math import gcd

print(gcd(7**2 - 1, 15), gcd(7**2 + 1, 15))   # 3, 5
```

10. Estimate how many logical qubits would be required to factor a 2048-bit RSA modulus, and discuss the timeline risk.

Test it

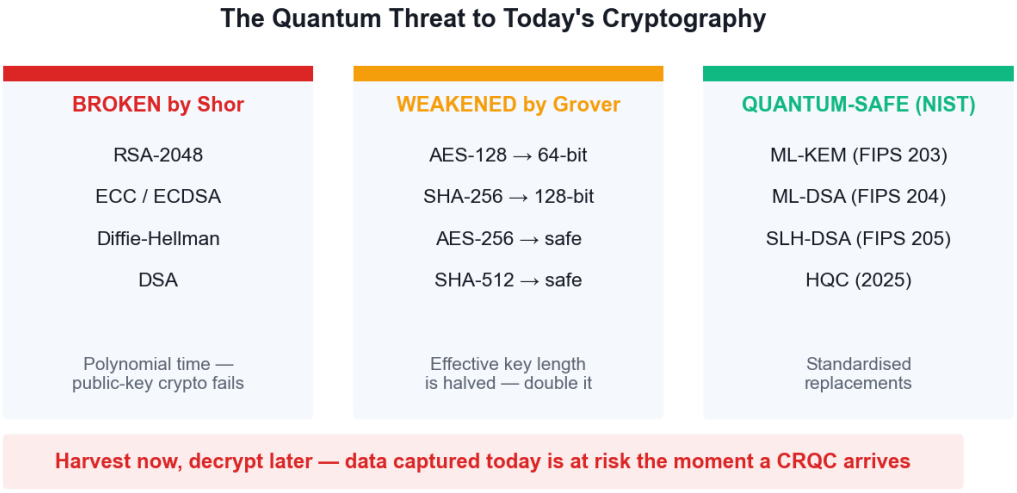
The measured phase yields period r = 4 for a = 7 and N = 15, and the classical post-processing step recovers the correct factors 3 and 5.

Note: The reference implementation for this activity is in labs/shor-algorithm/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 14 — Post-Quantum Cryptography Migration Assessment

Learning outcome: LO4, LO7 — Evaluate quantum applicability in the business context.

Goal: Assess which cryptography in a realistic system is broken by a quantum computer, map each algorithm to its NIST post-quantum replacement, and produce a prioritised migration plan against the harvest-now-decrypt-later risk.



What breaks, what is weakened, and the NIST replacements.

What you'll build

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

A completed cryptographic inventory and a prioritised PQC migration plan for a sample organisation. (Tools: NIST FIPS 203/204/205, OpenSSL, Python.)

Step-by-step

1. List the cryptography used in a typical enterprise: TLS certificates, VPNs, code signing, database encryption and backups.
2. Inspect a real certificate to identify its public-key algorithm and key size.

```
openssl s_client -connect www.tertiarycourses.com.sg:443 </dev/null
2>/dev/null | openssl x509 -noout -text | grep -A2 'Public Key Algorithm'
```

3. Classify each algorithm as broken by Shor (RSA, ECC, Diffie-Hellman) or merely weakened by Grover (AES, SHA-2).
4. Record that Grover halves the effective key length, so AES-128 falls to 64-bit security while AES-256 remains safe.
5. Map each broken algorithm to its NIST replacement: ML-KEM for key exchange, ML-DSA or SLH-DSA for signatures.
6. Note the standards: FIPS 203 (ML-KEM/Kyber), FIPS 204 (ML-DSA/Dilithium) and FIPS 205 (SLH-DSA/SPHINCS+).
7. Score each asset for harvest-now-decrypt-later risk based on how long its data must stay confidential.
8. Identify which systems need crypto-agility first — long-lived secrets and long-lived devices rank highest.
9. Produce a prioritised migration table listing asset, current algorithm, PQC replacement, risk score and target date.
10. Recommend hybrid classical-plus-PQC key exchange as the safe transitional deployment strategy.

Test it

Your inventory correctly separates Shor-broken asymmetric algorithms from Grover-weakened symmetric ones, maps each to the correct NIST FIPS standard, and ranks migration priority by how long the data must remain confidential.

Note: The reference implementation for this activity is in labs/pqc-migration/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 07 — Quantum Computing Applications and Use Cases

Finance · Healthcare · Science

Key concepts

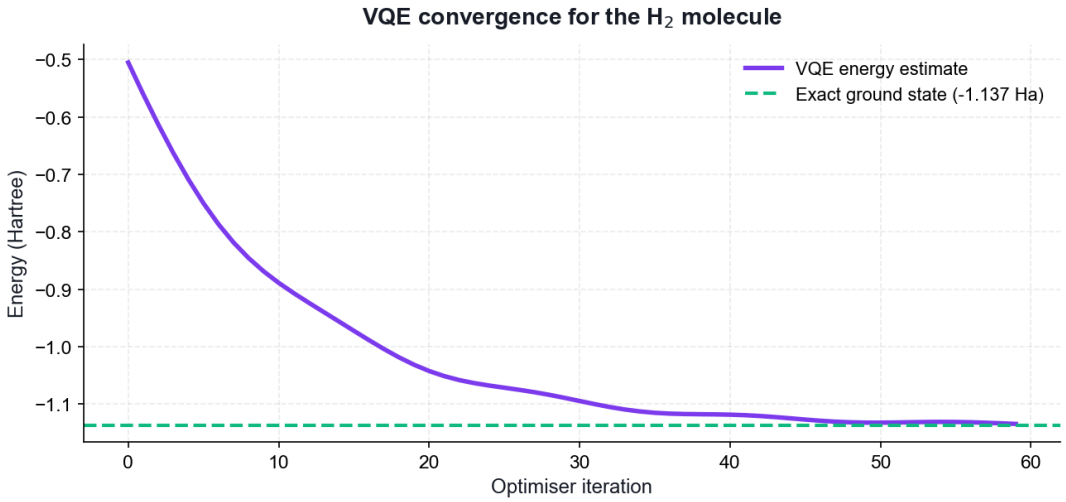
© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Finance — Portfolio optimisation, derivative pricing via amplitude estimation, credit risk and fraud detection are the leading finance use cases.
- Healthcare and life sciences — Molecular simulation with VQE, protein folding and accelerated drug discovery target problems classical HPC cannot reach.
- Science and materials — Simulating catalysts, superconductors, nitrogen fixation and battery chemistry is quantum computing's most natural application.
- Optimisation and logistics — QAOA addresses routing, scheduling and Max-Cut style combinatorial problems across supply chains and energy grids.
- Quantum machine learning — Quantum kernels and variational classifiers explore feature spaces that are expensive to compute classically.
- Assessing business value — Evaluate a use case on problem structure, data size, required qubit count and whether a classical method already suffices.

Activity 15 — Molecular Simulation with VQE (Healthcare & Science)

Learning outcome: LO4, LO7 — Evaluate quantum computing applicability in the business context.

Goal: Run the Variational Quantum Eigensolver to compute the ground-state energy of the hydrogen molecule — the flagship near-term use case for drug discovery, catalysis and materials science.



VQE convergence toward the H2 ground-state energy.

What you'll build

A VQE run that converges to the H2 ground-state energy, with an energy-versus-iteration convergence plot. (Tools: Qiskit 2.x, SciPy optimiser, matplotlib, IBM Quantum.)

Step-by-step

1. Open the vqe lab folder and review the ansatz and Hamiltonian definitions in main.py.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
cd labs/vqe && cat main.py
```

2. Understand the hybrid loop: a parameterised quantum circuit prepares a trial state, a classical optimiser updates the parameters.
3. Inspect the H2 Hamiltonian expressed as a weighted sum of Pauli strings.
4. Build the parameterised ansatz circuit whose rotation angles are the variational parameters.
5. Define the cost function that measures the expectation value of the Hamiltonian for a given parameter set.
6. Run the classical optimiser to minimise the energy over the parameters.

```
python labs/vqe/main.py
```

7. Plot energy against iteration and confirm the value converges downward toward the ground state.
8. Compare the converged energy against the known exact H2 ground-state energy of about -1.137 Hartree.
9. Explain why VQE suits today's noisy hardware — shallow circuits and a classical optimiser tolerate some noise.
10. Discuss which pharmaceutical and materials problems become tractable as qubit counts grow.

Test it

The VQE optimisation converges to an energy close to the exact H2 ground-state value of about -1.137 Hartree, and the convergence plot shows the energy decreasing monotonically toward that limit.

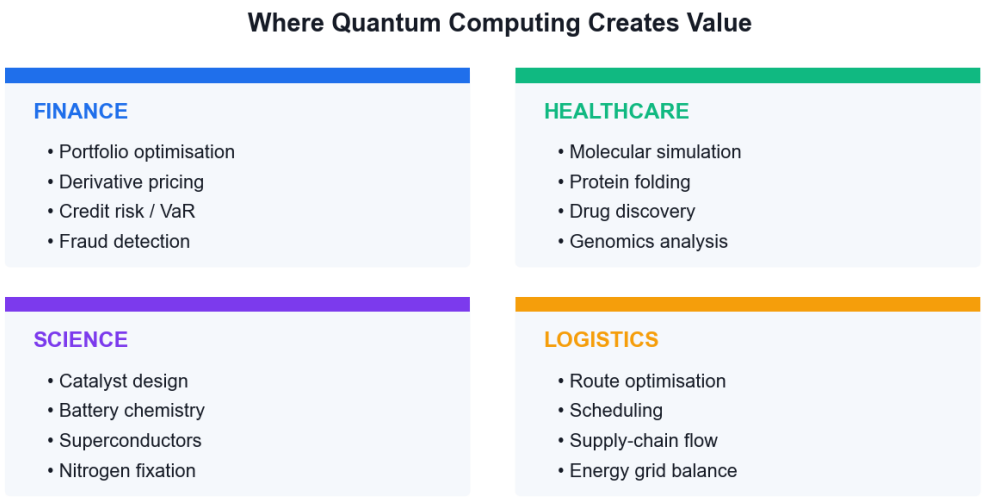
Note: The reference implementation for this activity is in labs/vqe/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 16 — Portfolio Optimisation with QAOA (Finance)

Learning outcome: LO4, LO6 — Evaluate applicability and select appropriate algorithms.

Goal: Apply the Quantum Approximate Optimization Algorithm to a Max-Cut style combinatorial problem, the same structure that underlies portfolio selection, routing and scheduling in financial services.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



Where QAOA-style optimisation creates value across sectors.

What you'll build

A QAOA run that finds the optimal cut on a small graph, with the solution quality compared against brute force. (Tools: Qiskit 2.x, NetworkX, SciPy, matplotlib.)

Step-by-step

1. Open the qaoa lab folder and review the Max-Cut problem definition in main.py.

```
cd labs/qaoa && cat main.py
```

2. Define a small graph whose vertices represent assets and whose edges carry correlation weights.
3. Encode the Max-Cut objective as a cost Hamiltonian built from ZZ terms on each edge.
4. Build the QAOA circuit alternating the cost Hamiltonian and the mixer Hamiltonian for p layers.
5. Initialise the register in equal superposition with Hadamard gates before the first layer.
6. Run the classical optimiser over the gamma and beta parameters to maximise the expected cut value.

```
python labs/qaoa/main.py
```

7. Sample the optimised circuit and identify the most frequent bit string as the proposed solution.
8. Brute-force all 2^n partitions classically and compare the optimal cut against the QAOA result.
9. Increase the number of layers p and observe how the approximation ratio improves.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

10. Map the result back to the finance framing: which assets fall on each side of the partition, and why that matters for diversification.

Test it

The most frequently sampled bit string from the optimised QAOA circuit corresponds to the true maximum cut found by brute force, and the approximation ratio improves as the number of layers p increases.

Note: The reference implementation for this activity is in labs/qaoa/. Read the source before you run it — understanding the circuit matters more than the output.

Topic 08 — Quantum Error Correction

Bit and Phase Flip Errors · QEC Algorithms · Fault Tolerance · The Future

Key concepts

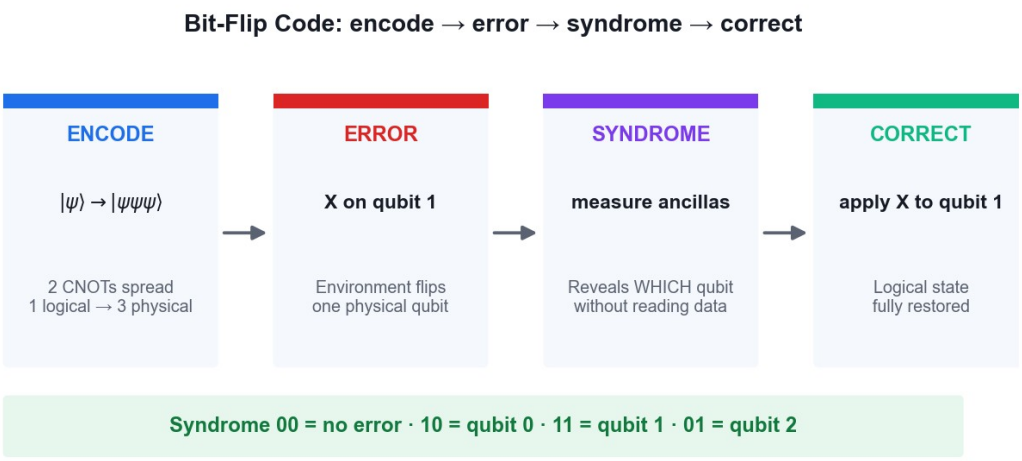
- Why QEC is needed — Qubits decohere and gates are imperfect; without correction, errors accumulate long before a useful algorithm completes.
- Quantum-specific challenges — No-cloning forbids simple copying, errors are continuous rather than discrete, and measurement destroys the state.
- Syndrome measurement — Ancilla qubits are measured to reveal WHICH error occurred without ever measuring — and collapsing — the logical state.
- Bit-flip and phase-flip codes — Three-qubit repetition codes correct X errors; conjugating with Hadamards turns phase flips into bit flips.
- Shor, Steane and surface codes — Shor's 9-qubit code corrects arbitrary single-qubit errors; Steane uses 7; surface codes are today's leading practical approach.
- Fault tolerance — Below the error threshold, concatenating codes suppresses errors faster than they accumulate — the path to useful quantum computers.

Activity 17 — Bit-Flip and Phase-Flip Error Correction Codes

Learning outcome: LO7 — Synthesize quantum methodologies to address complex problems.

Goal: Build the three-qubit repetition codes that detect and correct X and Z errors: encode a logical qubit, inject an error, measure the syndrome without collapsing the state, and apply the correction.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg



The bit-flip code cycle: encode, error, syndrome, correct.

What you'll build

Working bit-flip and phase-flip code circuits that correctly identify and repair an injected single-qubit error. (Tools: Qiskit 2.x, IBM Quantum Runtime, OpenQASM.)

Step-by-step

1. Open the quantum-error-correction lab and review the bit-flip code implementation.

```
cd labs/quantum-error-correction/bit-flip-code && cat main.py
```

2. Encode one logical qubit into three physical qubits using two CNOT gates.

```
qc = QuantumCircuit(5, 2)
```

```
qc.cx(0, 1); qc.cx(0, 2)
```

3. Inject a bit-flip error by applying an X gate to one of the three data qubits.

```
qc.x(1)   # simulate a bit-flip error on qubit 1
```

4. Compute the syndrome using two ancilla qubits and four CNOT gates comparing qubit pairs.

```
qc.cx(0, 3); qc.cx(1, 3); qc.cx(1, 4); qc.cx(2, 4)
```

5. Measure only the ancillas — this reveals which qubit flipped without measuring the logical state.

```
qc.measure([3, 4], [0, 1])
```

6. Decode the syndrome: 00 means no error, 10 means qubit 0, 11 means qubit 1 and 01 means qubit 2.
7. Apply the corrective X gate to the qubit identified by the syndrome and confirm the state is restored.
8. Run the bit-flip code on IBM Quantum hardware and verify the correction succeeds.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
python labs/quantum-error-correction/bit-flip-code/main.py
```

9. Move to the phase-flip code and note that conjugating with Hadamards turns Z errors into X errors.

```
cd ../phase-flip-code && python main.py
```

10. Explain why measuring the syndrome rather than the data qubits is what makes quantum error correction possible at all.

Test it

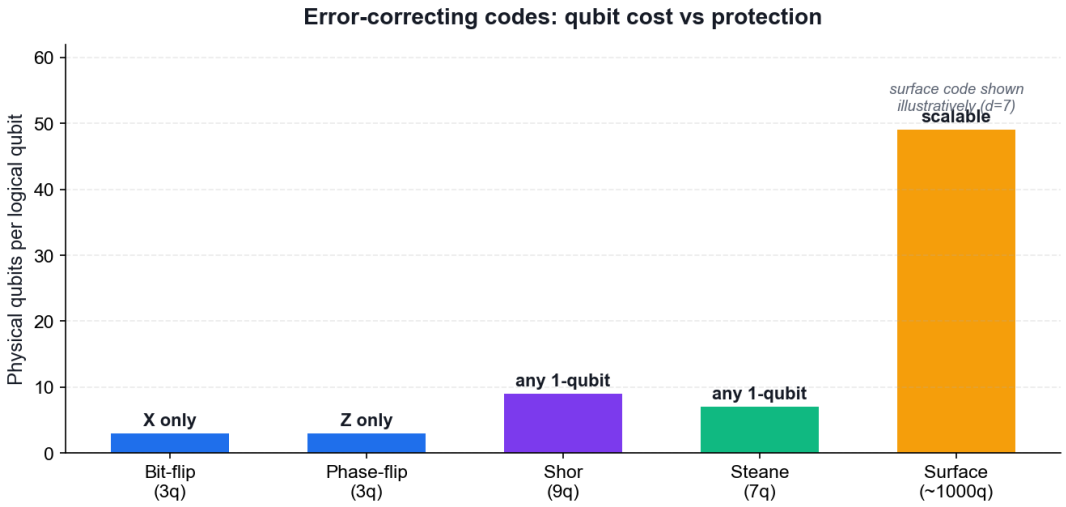
The syndrome measurement correctly identifies which qubit carried the injected error without collapsing the logical state, and applying the indicated correction restores the original logical qubit.

Note: The reference implementation for this activity is in labs/quantum-error-correction/. Read the source before you run it — understanding the circuit matters more than the output.

Activity 18 — Shor 9-Qubit and Steane 7-Qubit Codes

Learning outcome: LO7 — Synthesize quantum methodologies to address complex problems.

Goal: Step up to codes that correct arbitrary single-qubit errors: Shor's 9-qubit concatenated code and Steane's more efficient 7-qubit CSS code, then compare their qubit overhead and discuss the road to fault tolerance.



Qubit cost versus protection for the main error-correcting codes.

What you'll build

Working Shor 9-qubit and Steane 7-qubit code circuits, with a comparison of qubit cost against protection offered. (Tools: Qiskit 2.x, IBM Quantum Runtime, OpenQASM.)

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Step-by-step

1. Open the shor-9qubit-code folder and review how it concatenates the phase-flip and bit-flip codes.

```
cd labs/quantum-error-correction/shor-9qubit-code && cat main.py
```

2. Understand the structure: one logical qubit becomes three blocks of three, protecting against both X and Z errors.
3. Encode the logical qubit across all nine physical qubits.
4. Inject an arbitrary single-qubit error — X, Z or Y — into any one of the nine qubits.
5. Measure the syndrome and confirm the code identifies and corrects the error whichever type it was.

```
python main.py
```

6. Move to the steane-7qubit-code folder and review the CSS construction based on the Hamming code.

```
cd ../steane-7qubit-code && cat main.py
```

7. Run the Steane code and confirm it also corrects an arbitrary single-qubit error using only seven qubits.

```
python main.py
```

8. Compare the overhead of the two codes and note why Steane is more efficient than Shor.
9. Read about surface codes and why their 2D nearest-neighbour layout suits real superconducting hardware.
10. Discuss the error threshold: below it, concatenating codes suppresses errors faster than they accumulate.

Test it

Both the Shor 9-qubit and Steane 7-qubit circuits detect and correct an arbitrary injected single-qubit error (X, Y or Z), and you can state the qubit overhead each code requires.

Note: The reference implementation for this activity is in labs/quantum-error-correction/. Read the source before you run it — understanding the circuit matters more than the output.

Quantum Gate Reference

Use this table as a quick reference during the activities and the open-book assessment.

- X (Pauli-X) — the quantum NOT gate; maps |0⟩ to |1⟩ and |1⟩ to |0⟩. Self-inverse and Hermitian.
- Y (Pauli-Y) — combines a bit flip and a phase flip; equal to iXZ.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- Z (Pauli-Z) — the phase flip; leaves |0⟩ unchanged and maps |1⟩ to −|1⟩.
- H (Hadamard) — creates superposition; maps |0⟩ to |+⟩ and |1⟩ to |−⟩. Self-inverse.
- S and T — phase gates applying π/2 and π/4 respectively; S is the square root of Z, T of S.
- Rx, Ry, Rz — arbitrary rotations about the x, y and z axes of the Bloch sphere.
- CNOT (CX) — flips the target qubit if the control is |1⟩; the workhorse entangling gate.
- CZ — applies a phase of −1 only to the |11⟩ component; symmetric in its two qubits.
- SWAP — exchanges the states of two qubits; often needed to satisfy hardware connectivity.
- Toffoli (CCX) — flips the target only if BOTH controls are |1⟩; makes classical logic reversible.

Algorithm Selection Guide

Use this decision guide to choose an appropriate quantum algorithm — this directly supports LO6.

- Deciding a global property of a black-box function (constant vs balanced) — use Deutsch or Deutsch-Jozsa.
- Recovering a hidden linear structure or bit string — use Bernstein-Vazirani.
- Searching an unstructured space with no exploitable structure — use Grover for a quadratic speedup.
- Extracting periodicity or an eigenvalue phase — use the Quantum Fourier Transform or phase estimation.
- Factoring integers or computing discrete logarithms — use Shor's algorithm.
- Finding the ground-state energy of a molecule on noisy hardware — use VQE.
- Solving a combinatorial optimisation problem on noisy hardware — use QAOA.
- If a classical algorithm already solves the problem efficiently, quantum computing adds no advantage — this judgement is itself an assessable skill.

Post-Quantum Cryptography Quick Reference

- Broken by Shor's algorithm: RSA, Diffie-Hellman, DSA, ECC and ECDSA — all public-key cryptography based on factoring or discrete logarithms.
- Weakened by Grover's algorithm: symmetric ciphers and hash functions lose half their effective key length, so AES-128 falls to 64-bit security while AES-256 remains safe.
- FIPS 203 — ML-KEM (formerly CRYSTALS-Kyber), the standard for key encapsulation.
- FIPS 204 — ML-DSA (formerly CRYSTALS-Dilithium), the primary standard for digital signatures.
- FIPS 205 — SLH-DSA (formerly SPHINCS+), a hash-based signature alternative.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- HQC — selected by NIST in 2025 as a backup key-encapsulation mechanism based on error-correcting codes.
- Migration strategy: inventory your cryptography, prioritise long-lived secrets, and deploy hybrid classical-plus-post-quantum key exchange during the transition.

Assessment Preparation

- The Written Assessment (WA) is 1 hour of open-ended short-answer questions covering all eight topics.
- The Practical Performance (PP) is 1 hour of hands-on quantum circuit and algorithm tasks.
- Both are OPEN BOOK — you may use the course slides, this Learner Guide and approved materials.
- Re-run every activity from a blank notebook until you can build the circuits without referring back.
- Be able to explain, in your own words, why each algorithm achieves its speedup.
- Practise stating which quantum principle each protocol depends on — no-cloning, entanglement, interference or phase kickback.
- Remember to complete the TRAQOM survey and the Assessment digital attendance on assessment day.

Glossary

- **Qubit —** The basic unit of quantum information — a two-level system that can hold a superposition of $|0\rangle$ and $|1\rangle$.
- **Superposition —** A quantum state that is a weighted combination of basis states, described by complex amplitudes.
- **Amplitude —** The complex number whose squared magnitude gives the probability of a measurement outcome.
- **Born rule —** The rule that the probability of outcome x is the squared magnitude of its amplitude, $|\langle x|\psi\rangle|^2$.
- **Entanglement —** A correlation between qubits so strong that the joint state cannot be factored into individual states.
- **Bell state —** A maximally entangled two-qubit state, created by a Hadamard followed by a CNOT.
- **GHZ state —** The n-qubit generalisation of the Bell state, entangling three or more qubits.
- **Hilbert space —** The complex vector space in which quantum states live; n qubits span 2^n dimensions.
- **Unitary operator —** A reversible, norm-preserving operator ($U^\dagger U = I$); every quantum gate is unitary.
- **Hermitian operator —** A self-adjoint operator ($A^\dagger = A$) with real eigenvalues; represents an observable.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

- **Dirac notation —** Bra-ket notation: $|\psi\rangle$ for a state (ket) and $\langle\psi|$ for its dual (bra).
- **Bloch sphere —** The geometric representation of a single qubit's pure state as a point on a unit sphere.
- **Phase kickback —** The transfer of an eigenvalue phase from a target qubit back onto the control qubit.
- **Oracle —** A black-box unitary encoding the function an algorithm queries, usually as $|x\rangle|y\rangle \to |x\rangle|y \oplus f(x)\rangle$.
- **No-cloning theorem —** The result that no unitary can copy an arbitrary unknown quantum state.
- **Quantum teleportation —** Transferring an unknown qubit state using a shared Bell pair and two classical bits.
- **Superdense coding —** Transmitting two classical bits by sending one qubit of a pre-shared entangled pair.
- **QKD —** Quantum Key Distribution — establishing a shared secret key whose security rests on physics.
- **BB84 —** The original QKD protocol, using two conjugate measurement bases to detect eavesdropping.
- **Grover's algorithm —** Unstructured search in $O(\sqrt{N})$ queries via amplitude amplification.
- **Shor's algorithm —** Polynomial-time integer factoring by reducing it to quantum period finding.
- **QFT —** Quantum Fourier Transform — the quantum analogue of the discrete Fourier transform.
- **VQE —** Variational Quantum Eigensolver — a hybrid algorithm for finding ground-state energies.
- **QAOA —** Quantum Approximate Optimization Algorithm — a hybrid algorithm for combinatorial optimisation.
- **NISQ —** Noisy Intermediate-Scale Quantum — the current era of quantum hardware without full error correction.
- **Decoherence —** The loss of quantum information as a qubit interacts with its environment.
- **Syndrome measurement —** Measuring ancilla qubits to detect an error without collapsing the logical state.
- **Surface code —** A leading quantum error-correcting code using a 2D lattice of nearest-neighbour qubits.
- **Fault tolerance —** Computing reliably with imperfect components by keeping error rates below a threshold.
- **PQC —** Post-Quantum Cryptography — classical algorithms believed secure against quantum attack.
- **CRQC —** Cryptographically Relevant Quantum Computer — one large enough to break RSA or ECC.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg